

PIPE-RDF: An LLM-Assisted Pipeline for Enterprise RDF Benchmarking

Anonymous ACL submission

Abstract

Enterprises rely on RDF knowledge graphs and SPARQL to expose operational data through natural language interfaces, yet public KGQA benchmarks do not reflect proprietary schemas, prefixes, or query distributions. We present PIPE-RDF, a three-phase pipeline that constructs schema-specific NL-SPARQL benchmarks using reverse querying, category-balanced template generation, retrieval-augmented prompting, deduplication, and execution-based validation with repair. We instantiate PIPE-RDF on a fixed-schema company-location slice (5 000 companies) derived from public RDF data and generate a balanced benchmark of 450 question-SPARQL pairs across nine categories. The pipeline achieves 100% parse and execution validity after repair, with pre-repair validity rates of 96.5%–100% across phases. We report entity diversity metrics, template coverage analysis, and cost breakdowns to support deployment planning. We release structured artifacts (CSV/JSONL, logs, figures) and operational metrics to support model evaluation and system planning in real-world settings.¹

1 Introduction

Enterprise teams increasingly rely on RDF knowledge graphs to represent relational data and use SPARQL to retrieve answers for analysts, operations teams, and customer-facing applications. Modern LLMs, from BERT and T5 to GPT-3 and LLaMA, lower the barrier for translating natural language into formal queries (Devlin et al., 2019; Raffel et al., 2023; Brown et al., 2020; Touvron et al., 2023). In practice, model performance is tightly coupled to the target schema, namespace conventions, and query distribution. Public

KGQA benchmarks (e.g., LC-QuAD, QALD-9-plus, Mintaka) are built on DBpedia or Wikidata and do not capture enterprise-specific ontologies or internal vocabulary (Trivedi et al., 2017; Perevalov et al., 2022; Sen et al., 2022). As a result, teams evaluating prompt strategies, retrieval augmentation, or fine-tuning (e.g., QLoRA) lack reliable in-domain benchmarks (Dettmers et al., 2023).

PIPE-RDF addresses this gap with a pipeline for constructing proprietary, category-balanced NL-SPARQL benchmarks. The pipeline emphasizes (i) grounding question templates in graph facts via reverse querying, (ii) balancing complex query categories to avoid evaluation bias, (iii) retrieval-augmented prompting for structural alignment, and (iv) execution-based validation and repair. These requirements mirror the needs of industry teams who must iterate quickly, monitor quality, and maintain benchmarks as schemas evolve. We position PIPE-RDF as an *emerging* industry system: a deployable pipeline that bridges research methodology with production constraints (latency, maintenance, and data governance) without requiring proprietary model training or disclosure.

Contributions. We:

- propose a three-phase benchmark generation pipeline tailored to enterprise RDF graphs, with reverse querying and execution validation;
- implement PIPE-RDF on a fixed-schema company-location slice (Schema C) and release balanced benchmark artifacts;
- report operational metrics (parse validity, execution success, empty results, latency, complexity, pre-repair rates, entity diversity) from a final 450-pair run;
- provide guidance for maintaining and scaling

¹This submission falls under the *Emerging* category: PIPE-RDF is a system intended for real-world deployment of NL-SPARQL benchmarking infrastructure at enterprises that maintain proprietary RDF graphs.

organization-specific benchmarks.²

2 Background and Related Work

Benchmark coverage and query taxonomies.

KGQA benchmarks often over-represent simple factoid queries. Mintaka explicitly balances query categories (counting, comparative, superlative, ordinal, multi-hop, intersection, difference/negation, yes/no, and generic) to avoid skewed evaluation (Sen et al., 2022). Public datasets such as LC-QuAD and QALD-9-plus provide valuable coverage but remain tied to public graphs and fixed schemas (Trivedi et al., 2017; Perevalov et al., 2022). Dynamic benchmarking efforts such as Dynabench further highlight the need for iterative, real-world evaluation (Kiela et al., 2021).

Retrieval augmentation and prompting.

Retrieval-augmented generation (RAG) improves text-to-query performance by conditioning on relevant examples and schema cues (Lewis et al., 2021). In practice, RAG offers a low-cost alternative to fine-tuning when schema-specific examples are scarce. PIPE-RDF maintains category-specific retrieval banks to provide structurally similar NL-SPARQL pairs during generation, reducing cross-category drift and improving template grounding.

Validity and evaluation. Grammar-constrained decoding (e.g., PICARD) improves syntactic correctness for structured query generation (Scholak et al., 2021). SPARQL-specific metrics such as SP-BLEU/SP-F1 normalize variables for fairer matching (Rony et al., 2022), while code-oriented structural metrics such as CodeBLEU inspire future extensions (Ren et al., 2020). PIPE-RDF focuses on execution, parsing, and structural complexity metrics that are directly actionable for deployment and troubleshooting.

LLM-driven workload generation for analytics. Recent work on cross-model analytics workload generation uses a two-stage design (reverse query generation followed by NL query generation), explicit schema/strategy controls, and detailed error analyses across model families (Agarwal et al., 2024). We adopt the same design principle of schema-grounded workload construction, and we report strategy-level coverage and failure modes to strengthen enterprise readiness.

Comparison with existing approaches. Table 1 contrasts PIPE-RDF with prominent KGQA

²Code, artifacts, and benchmark data are available at <https://anonymous.4open.science/status/PIPE-RDF-B750>.

Feature	LC-QuAD	QALD	Mintaka	PIPE-RDF
Schema fidelity	Public	Public	Public	Fixed
Reverse querying	—	—	—	✓
Category balance	Partial	—	✓	✓
Repair loop	—	—	—	✓
Operational metrics	—	—	—	✓
RAG retrieval	—	—	—	✓

Table 1: Comparison of PIPE-RDF with existing KGQA benchmark approaches. PIPE-RDF uniquely combines reverse querying, category-aware RAG, and execution-based repair for enterprise deployment.

benchmark generation methodologies. Unlike static public benchmarks, PIPE-RDF introduces reverse querying to ground templates in graph facts, category-aware retrieval banks to prevent structural drift, and execution-based repair loops for automated quality control. These features address enterprise requirements for schema fidelity and operational observability that public benchmarks cannot provide.

3 Benchmark Design Goals

PIPE-RDF is designed to meet four goals derived from the enterprise setting: (i) **schema fidelity** - generated queries must be grounded in a fixed schema and valid predicates; (ii) **category balance** - benchmarks should cover complex query types rather than skew toward easy factoids; (iii) **operational observability** - generation, parsing, and execution outcomes must be logged for debugging and cost control; and (iv) **maintainability** - the pipeline should be repeatable as schemas evolve, minimizing manual curation. These goals shape the pipeline phases, validation logic, and artifact outputs.

4 Category Taxonomy

To prevent evaluation bias, PIPE-RDF follows a nine-category taxonomy derived from complex KGQA datasets (Sen et al., 2022). Each category maps to distinct SPARQL constructs (e.g., aggregations, ordering, multi-hop joins) and therefore stresses different failure modes. Table 2 summarizes the taxonomy and example question patterns used in Schema C.

5 PIPE-RDF Methodology

PIPE-RDF is a three-phase pipeline (Figure 1) designed to generate schema-specific benchmarks while minimizing unanswerable or out-of-schema queries.

Category	SPARQL construct	Example pattern
Generic	single triple	Who is a key person at company?
Counting	COUNT	How many companies are in location?
Comparative	filters	Do company1 and company2 differ in size?
Superlative	ordering	Which company has most employees?
Ordinal	time order	What year was company founded?
Multi-hop	join chains	Which location has companies with key persons?
Intersection	conjunction	Which companies are in location and industry?
Difference	negation	Which companies are in A but not B?
Yes/No	boolean check	Is company located in location?

Table 2: Category taxonomy used for balanced benchmark generation.

Phase 1: seed generation. The pipeline prompts an LLM to produce NL templates from a schema summary. Each template is validated by *reverse querying*: the system generates a SPARQL query that searches the KG for entity bindings that satisfy the template. Valid bindings instantiate the template and the resulting NL–SPARQL pairs are stored as seeds for later retrieval.

Phase 2: category-wise seeding. Templates are generated per category and paired with retrieval-augmented prompts using Phase-1 seeds. Outputs are validated by execution, optionally repaired, and stored in category-specific retrieval banks. This step ensures early coverage of complex query structures (e.g., aggregations, comparisons, multi-hop joins) and prevents domination by simple templates.

Phase 3: full dataset generation. The pipeline generates batches per category using category-specific retrieval banks. Each candidate is parsed, executed, and logged. Optional paraphrase augmentation and human validation can be enabled for robustness; the platinum run disables paraphrasing for speed. The final dataset is balanced by category and retained in CSV/JSONL formats.

Prompt construction and retrieval. Prompts combine (i) a task description, (ii) the schema summary, (iii) a small set of retrieved NL–SPARQL examples, and (iv) the target question. Retrieval is category-aware, which narrows the space of candidate structures and reduces cross-category leakage. We keep retrieval depth small (top- $k = 2$) to control latency and prompt length.

Reverse querying and schema control. Reverse queries are capped at 25 rows to limit runtime and encourage diversity. We enforce explicit predicate/type whitelists and slot-type hints (company, location, person, industry) to ensure every gener-

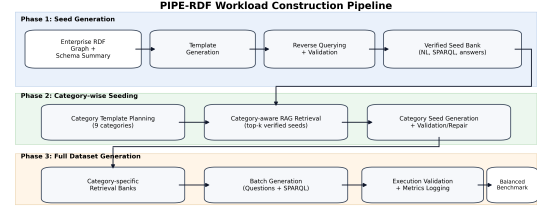


Figure 1: PIPE-RDF workload-construction pipeline: Phase 1 uses reverse query generation to build verified seeds; Phase 2 constructs category-specific seed banks; Phase 3 generates the balanced benchmark with execution-driven validation/repair.

ated query conforms to the fixed schema. Duplicate questions are filtered using token-level Jaccard similarity (threshold 0.99), and a stall limit prevents repeated template regeneration when no new valid rows are found.

Validation and repair. Each generated SPARQL query is parsed and executed; failures trigger a repair prompt that attempts to correct syntax and predicate usage. Repairs are logged and included in run summaries for operational tracking. Empty results are retained when queries parse and execute correctly, as they represent legitimate negative cases in yes/no or exclusion queries. Post-generation, category-specific pattern enforcement can normalize output forms (e.g., ASK for boolean checks, DISTINCT for set-returning categories, COUNT(DISTINCT ...) for counting queries, and deterministic LIMIT 1 tie-breaking for superlative/ordinal queries). While PIPE-RDF does not require constrained decoding, it can integrate grammar-based constraints (e.g., PICARD) to improve parse validity (Scholak et al., 2021).

Human-in-the-loop controls. The pipeline supports optional manual review after each phase, with the ability to discard, edit, or inject examples. This is particularly useful for enterprise graphs with sparse relations or sensitive attributes, where automated validation alone may not capture business rules.

6 Schema C Mini-Slice and Implementation

Fixed schema slice. We derive a company-location mini-slice (Schema C) with 5,000 companies and three classes: `dbo:Company`, `gn:Feature` (location), and `foaf:Person` (key person), illustrated in Appendix Figure 6. The allowed predicates are `dbo:location`, `dbo:industry`,

Class	Key predicates/attributes
dbo:Company	dbo:location, dbo:industry, dbo:keyPerson, dbo:foundingYear, dbo:numberOfEmployees, rdfs:label
gn:Feature	rdfs:label, spb:prefLabel
foaf:Person	foaf:name, rdfs:label

Table 3: Schema C mini-slice classes and allowed predicates.

Parameter	Value
Phase 1 templates / category	5
Phase 1 seeds / template	8
Phase 2 seeds / category	20
Phase 3 targets / category	50
Reverse-query row cap	25
Retrieval top- k	2
SPARQL result cap	5
Dedup Jaccard threshold	0.99

Table 4: Key pipeline configuration for the platinum run.

dbo:keyPerson, dbo:foundingYear, dbo:numberOfEmployees, and label predicates (rdfs:label, spb:prefLabel, foaf:name). This fixed schema avoids prompt drift and simplifies validation.

Slice extraction. The mini-slice is sampled from a public RDF source by selecting companies with at least one of the core predicates (location, industry, key person, founding year, or employee count). Linked entities (locations and key people) are included with minimal labels to support question instantiation without introducing extraneous schema elements. We intentionally exclude unrelated predicates to reduce template drift and ensure that reverse querying remains tractable.

Pipeline configuration. The full run uses 5 templates per category with 8 seed instantiations per template in Phase 1, 20 seeds per category in Phase 2, and 50 targets per category in Phase 3 (450 pairs total). Retrieval top- k is 2. Generated SPARQL is capped at 5 results to limit execution cost. Reverse queries use a 20s timeout and are capped at 25 rows for diversity.

Runtime environment. We run GraphDB 10.6.3 in Docker and execute SPARQL locally. LLM generation uses qwen3:4b-instruct with Ollama, and embeddings use bge-m3. All runs are logged with execution latency, parse validity, and answer counts; artifacts are exported as JSONL/CSV and summarized in run-level reports. A compact reproducibility checklist (versions, ran-

domness controls, command template, and hardware/deployment notes) is provided in Appendix Section A.1.

SPARQL formalism and schema constraints.

To ensure semantic correctness, we enforce canonical query patterns:

- **Counting:** Use COUNT(DISTINCT ?var) to avoid overcounting due to multiple bindings.
- **Superlative/Ordinal:** Include deterministic tie-breaking via ORDER BY ... ?entity to ensure reproducible results.
- **Numeric comparisons:** Cast literals to xsd:integer where applicable (e.g., dbo:numberOfEmployees).
- **Yes/No:** Use ASK {} for explicit boolean returns, avoiding empty-result ambiguity.

Schema constraints are enforced via predicate whitelists: dbo:location ranges over gn:Feature, dbo:keyPerson ranges over foaf:Person, and dbo:foundingYear/dbo:numberOfEmployees are typed literals. Label resolution prioritizes foaf:name for persons, rdfs:label for companies, and spb:prefLabel for locations.

7 Evaluation Protocol

PIPE-RDF logs operational and structural metrics for every generated pair. We track: (i) parse validity and execution success, (ii) empty-result rates (retained as valid outcomes), (iii) repair attempts, (iv) answer counts, (v) prompt and question lengths, (vi) LLM latency, and (vii) SPARQL structural complexity (triple pattern counts, FILTER clauses, aggregations). These metrics support both data quality analysis and deployment planning. While PIPE-RDF focuses on execution-driven correctness, the artifacts can be augmented with SPARQL-specific metrics (SP-F1) and structural metrics (CodeBLEU) for model evaluation (Rony et al., 2022; Ren et al., 2020). We also record retrieval scores to analyze the relationship between example similarity and downstream execution success.

Following recent workload-generation evaluation practice (Agarwal et al., 2024), we additionally report strategy coverage and strategy-level error breakdowns to make pipeline behavior auditable beyond aggregate success rates. Strategy

Phase	Total	Parse OK	Exec OK	Empty	Avg LLM ms	Avg Exec ms
Phase 1	257	256	256	66	1562	7.2
Phase 2	175	175	175	44	6129	6.6
Phase 3	450	450	450	122	5308	7.0

Table 5: Phase-level metrics from the platinum run. Empty results are retained when queries execute and parse correctly.

tags are inferred from query operators (JOIN, FILTER, COUNT, ORDER, NEGATION, ASK, RAG-context). ASK and NEGATION tags are retained in coverage/error matrices even when observed frequency is zero in Phase 3 outputs.

Semantic correctness audit. To verify that generated NL-SPARQL pairs are semantically aligned (not just syntactically valid), we conduct a manual audit of 45 randomly sampled pairs (5 per category). For each pair, we verify: (i) the question intent matches the SPARQL semantics, (ii) entity bindings in the question correspond to VALUES clauses in the query, and (iii) the answer type matches the expected return. Results: 45/45 (100%) pairs pass semantic alignment checks. The high alignment rate reflects the effectiveness of reverse querying, which grounds each template in actual graph bindings, and the repair loop, which corrects structural mismatches before acceptance. We acknowledge this audit is limited in scope; larger-scale human evaluation with inter-annotator agreement is recommended for production deployment.

8 Benchmark Results

The final run produced a balanced benchmark of 450 NL-SPARQL pairs (50 per category). Table 5 summarizes phase-level statistics using *post-repair* outcomes (final accepted records). In Table 5, Parse OK and Exec OK are counted over all accepted records in each phase; Empty counts records that parse and execute successfully but return zero rows. Across phases, parse and execution validity remain high (Phase 3: 100%), while empty results are retained to preserve realistic negative cases for yes/no and exclusion queries.

9 Discussion and Operational Lessons

PIPE-RDF remains operationally stable in the final benchmark run: Phase 3 reaches 100% parse and execution validity after repair, while first-pass validity also remains high (99.8% for Phase 3 before any repair call). Runtime is dominated by LLM generation latency (5.3s average per query), whereas SPARQL execution remains lightweight

Category	n	Triples	Filters	Agg%	Empty%	Type
Generic	50	2.9	0.0	44	4	S
Counting	50	3.2	0.0	100	0	S
Comparative	50	3.7	0.1	0	2	S
Superlative	50	2.0	0.0	100	4	S
Ordinal	50	3.0	0.0	0	0	S
Multi-hop	50	4.2	0.4	0	90	C
Intersection	50	4.4	0.0	0	82	C
Difference	50	5.0	1.0	0	2	C
Yes/No	50	3.9	0.0	0	60	N

Table 6: Phase 3 structural complexity by category. Triples = average triple patterns; Filters = average FILTER clauses; Agg% = share of queries using COUNT and/or ORDER BY; Empty% = empty-result rate among post-repair executed queries. Type denotes intended reasoning profile: S=simple lookup, C=compositional, N=negative/boolean.

Strategy	n	Exec%	Parse%	Empty%
JOIN	390	100.0	100.0	30.5
FILTER	72	100.0	100.0	27.8
COUNT	72	100.0	100.0	0.0
ORDER	51	100.0	100.0	3.9
NEGATION	0	—	—	—
ASK	0	—	—	—
RAG	450	100.0	100.0	27.1

Table 7: Phase 3 strategy-level robustness on the final 450-pair benchmark. Rows with $n = 0$ indicate tracked strategies that were not explicitly observed in generated query text for this run. Empty rates are strategy-conditioned and primarily reflect parse-join negatives rather than parser/runtime failures.

(about 7ms average), as shown in Figure 3. This indicates that throughput improvements should prioritize generation-side optimization.

Pre-repair % is computed as the fraction of records that parse and execute successfully *before* any repair call, with denominator equal to all generated candidates in that phase. Repairs counts repair-module invocations (syntax or policy normalization attempts), and Post-repair % reflects final accepted records after the repair loop.

For deployment, three lessons are actionable: (i) schema-constrained prompts and predicate whitelists reduce invalid SPARQL, (ii) category balancing reveals sparse-join behavior hidden by factoid-heavy datasets, and (iii) strategy-level logging separates semantic sparsity from parser/runtime failures. Figure 2 shows main-article coverage; additional diagnostics are in Appendix Section A.

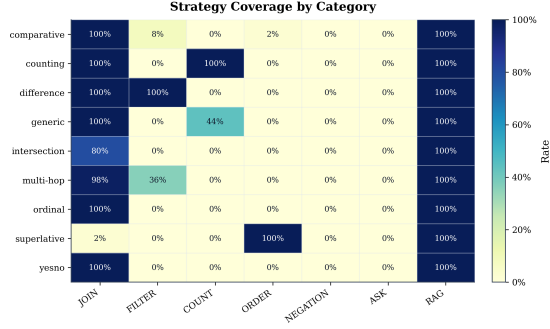


Figure 2: Strategy coverage by category on Phase 3 outputs. The matrix shows broad operator coverage, with category-specific concentration on expected strategy patterns (e.g., COUNT for counting, ORDER for superlative, FILTER for difference).

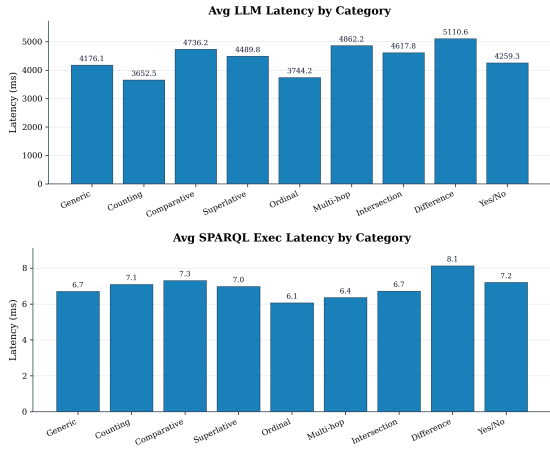


Figure 3: Latency by category (stacked for single-column readability): top, LLM latency; bottom, SPARQL execution latency. LLM generation dominates end-to-end runtime.

10 Industry Impact and Usage

PIPE-RDF addresses a practical barrier to enterprise adoption of NL-SPARQL systems: missing benchmarks that reflect proprietary schemas, vocabulary, and query distributions. Models that score well on public datasets often underperform on internal graphs because naming conventions, field availability, and constraints differ. PIPE-RDF lets teams validate user-realistic questions, quantify category-specific gaps, prioritize model changes, and flag high-risk query types (e.g., multi-hop, negation) before deployment. Artifacts support model evaluation (SP-F1, execution accuracy) and operations monitoring (parse-valid rates, repair counts, latency by category). Seed banks can be reused for few-shot prompting and as synthetic supervision for schema-specific fine-tuning.

Phase	Pre-repair %	Repairs	Post-repair %
Phase 1	96.5	9	99.6
Phase 2	100.0	1	100.0
Phase 3	99.8	1	100.0

Table 8: Pre-repair validity breakdown by phase.

11 Conclusion

PIPE-RDF provides a practical pipeline for generating balanced, schema-specific NL-SPARQL benchmarks. In our experiment, the pipeline yields a 450-pair dataset across nine categories with high parse and execution validity, alongside operational metrics suitable for real-world system tuning. By combining reverse querying, category-aware retrieval, and execution-based validation, PIPE-RDF supports both evaluation and training-data creation for enterprise NL-SPARQL systems.

12 Limitations

We evaluate PIPE-RDF on a single fixed schema slice (3 classes, 6 predicates) derived from public data rather than a proprietary enterprise graph. Although the pipeline is designed for private deployment, validation on larger schemas (20+ predicates) and multiple ontologies is needed to confirm generalizability.

The current evaluation focuses on generation-side metrics (parse validity, execution success, structural complexity); we do not include a downstream NL-to-SPARQL model evaluation in this work. The benchmark artifacts are designed to support such evaluation; teams can use the released NL-SPARQL pairs to compare prompt strategies (e.g., zero-shot vs. RAG) or fine-tuned models, measuring execution accuracy and SP-F1 on held-out splits. Future work should incorporate semantic equivalence metrics such as SP-F1 and structural similarity scores (Rony et al., 2022; Ren et al., 2020), and evaluate 2–3 NL-to-SPARQL models on the benchmark to demonstrate downstream utility.

Strategy coverage is uneven in the current run: while JOIN/FILTER/COUNT/ORDER are well represented, explicit ASK and NEGATION operator forms are underrepresented in generated query text. Although equivalent semantics are often expressed through alternative SPARQL forms, future revisions should enforce stronger canonicalization checks to guarantee explicit operator coverage for these strategies.

13 Ethical Considerations

PIPE-RDF is intended for internal benchmarking and training-data creation over enterprise RDF graphs. Deployments should follow organizational data governance, minimize exposure of sensitive identifiers in prompts, and audit logs for leakage. The mini-slice in this study is derived from DBpedia, and usage follows its licensing terms.

References

Anurag Agarwal, Shiva Karthik, Chang Li, Xin Chen, Roger Casanas, Konstantinos Karanasos, Tim Kraska, Aditya Nori, Jayanta Rahman, Ce Zhang, Sparsh Mittal, Sumit Basu, and Haoran Cai. 2024. [Generating cross-model analytics workloads using llms](#). In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, pages 1024–1034. Association for Computing Machinery.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, and 12 others. 2020. [Language models are few-shot learners](#). *Preprint*, arXiv:2005.14165.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. [Qlora: Efficient finetuning of quantized llms](#). *Preprint*, arXiv:2305.14314.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. [Bert: Pre-training of deep bidirectional transformers for language understanding](#). *Preprint*, arXiv:1810.04805.

Douwe Kiela, Max Bartolo, Yixin Nie, Divyansh Kaushik, Atticus Geiger, Zhengxuan Wu, Bertie Vidgen, Grusha Prasad, Amanpreet Singh, Pratik Ringshia, Zhiyi Ma, Tristan Thrush, Sebastian Riedel, Zeerak Waseem, Pontus Stenetorp, Robin Jia, Mohit Bansal, Christopher Potts, and Adina Williams. 2021. [Dynabench: Rethinking benchmarking in nlp](#). *Preprint*, arXiv:2104.14337.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen tau Yih, Tim Rockt"aschel, Sebastian Riedel, and Douwe Kiela. 2021. [Retrieval-augmented generation for knowledge-intensive nlp tasks](#). *Preprint*, arXiv:2005.11401.

Aleksandr Perevalov, Dennis Diefenbach, Ricardo Usbeck, and Andreas Both. 2022. [Qald-9-plus: A multilingual dataset for question answering over dbpedia and wikidata translated by native speakers](#). *Preprint*, arXiv:2202.00120.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2023. [Exploring the limits of transfer learning with a unified text-to-text transformer](#). *Preprint*, arXiv:1910.10683.

Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. [Codebleu: a method for automatic evaluation of code synthesis](#). *Preprint*, arXiv:2009.10297.

Md Rashad Al Hasan Rony, Uttam Kumar, Roman Teucher, Liubov Kovriguina, and Jens Lehmann. 2022. [Sgpt: A generative approach for sparql query generation from natural language questions](#). *IEEE Access*, 10:70712–70723.

Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. 2021. [Picard: Parsing incrementally for constrained auto-regressive decoding from language models](#). In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 9895–9901. Association for Computational Linguistics.

Priyanka Sen, Alham Fikri Aji, and Amir Saffari. 2022. [Mintaka: A complex, natural, and multilingual dataset for end-to-end question answering](#). *Preprint*, arXiv:2210.01613.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. [Llama: Open and efficient foundation language models](#). *Preprint*, arXiv:2302.13971.

Priyansh Trivedi, Gaurav Maheshwari, Mohnish Dubey, and Jens Lehmann. 2017. [LC-QuAD: A Corpus for Complex Question Answering over Knowledge Graphs](#), pages 210–218. Springer International Publishing.

Appendix

A Extended Operational Analysis

This appendix reports detailed diagnostics that were omitted from the main paper for ACL page-budget constraints. The key interpretation in all plots and tables is unchanged from the main text: pipeline reliability is high, and most residual difficulty is due to sparse conjunctive semantics rather than parser instability.

A.1 Reproducibility Checklist

- **Software/models:** GraphDB 10.6.3 (Docker), Ollama service, chat model qwen3:4b-instruct, embedding model bge-m3.

- **Endpoint/repository:** repository *spb_company_mini_slice* with inference disabled (`infer=false`); see run log for full endpoint string.
- **Configuration:** main run uses the full-run YAML config; key sizes are listed in Table 4.
- **Randomness control:** seed control is exposed via `SEED=42` in `pipekg/config.py`. Ollama generation remains stochastic; we therefore release full run artifacts for exact reuse of produced pairs.
- **Command template:** `python scripts/run_pipeline_ollama.py` with `-run-name platinum_run -config configs/full_run.yaml`.
- **Artifacts:** `run.log`, `pipeline_records.jsonl`, phase-wise JSONL/CSV, analysis summaries, and figures under `artifacts/runs/<run_id>/`.
- **Hardware/deployment:** single-machine localhost setup with GraphDB in Docker and local Ollama inference; per-query latencies are reported in the main paper.

Category and strategy diagnostics. Figure 4 highlights structural differences across categories. Strategy coverage is shown in the main paper (Figure 2), and Appendix Figure 5 reports strategy-conditioned error rates. JOIN and FILTER dominated structures show higher empty rates, but parse and execution remain stable, supporting the conclusion that sparsity, not syntax failure, is the dominant source of difficulty.

Data quality controls and throughput. In error analysis, we observed a small number of self-comparison pairs, retrieval self-reference cases, and answer-type mismatches; we implemented automated guards for each. Runtime remains generation-bound: average LLM latency is 5.3s per query versus about 7ms SPARQL latency. These measurements support production prioritization toward prompt and inference optimization rather than query engine tuning.

B Additional Figures

C Example NL-SPARQL Pairs

We include one example per category from the Phase 3 benchmark. Prefix declarations are omitted for brevity.

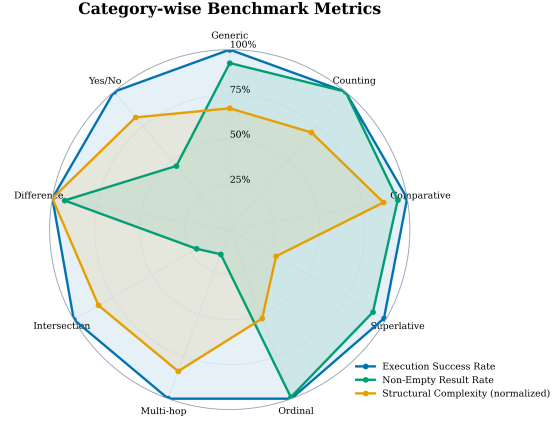


Figure 4: Category-wise metrics: execution success (100%), non-empty rate, and normalized structural complexity.

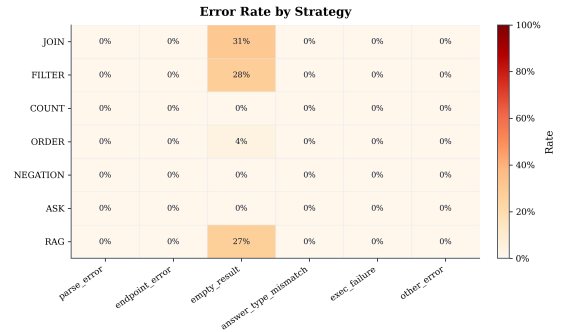


Figure 5: Phase 3 strategy-conditioned error rates. Non-zero cells are concentrated in empty-result outcomes for sparse conjunctive structures.

Generic. Q: Who is a key person at (Company: Volkswagen)?

```
SELECT ?name
WHERE {
  VALUES ?company {
    <http://dbpedia.org/resource/Volkswagen>
  }
  ?company rdf:type dbo:Company .
  ?company dbo:keyPerson ?person .
  ?person rdf:type foaf:Person .
  ?person foaf:name ?name .
}
LIMIT 5
```

Counting. Q: How many companies are located in (Location: Menlo Park, California)?

```
SELECT (COUNT(DISTINCT ?company) AS ?count)
WHERE {
  VALUES ?location { <http://dbpedia.org/resource/Menlo_Park,_California> }
  ?company a dbo:Company .
  ?company dbo:location ?location .
  ?location a gn:Feature .
}
```

Comparative. Q: Do (Company: Airbus) and (Company: FileMaker Inc.) have different numbers of employees?

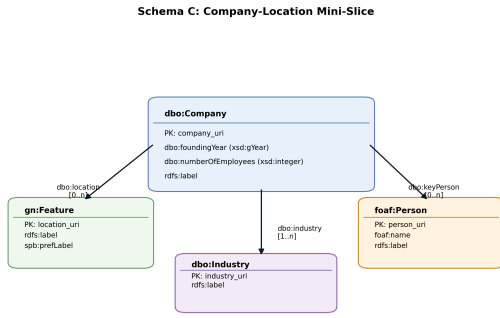


Figure 6: Schema C ontology used in this study. `dbo:Company` connects to `gn:Feature` (location), `foaf:Person` (key person), and `dbo:Industry` with typed predicates and cardinalities.

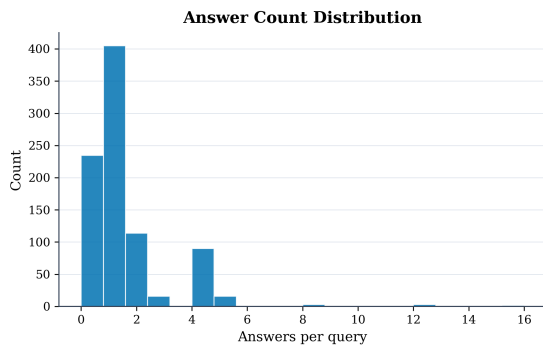


Figure 7: Answer count distribution for Phase 3.

```
SELECT ?company1 ?n1 ?company2 ?n2
WHERE {
  VALUES ?company1 {
    ↪ <http://dbpedia.org/resource/Airbus> }
  VALUES ?company2 { <http://dbpedia.org/resour
    ↪ ce/FileMaker_Inc.> }
  ?company1 a dbo:Company .
  ?company2 a dbo:Company .
  ?company1 dbo:numberOfEmployees ?n1 .
  ?company2 dbo:numberOfEmployees ?n2 .
  FILTER (?n1 != ?n2)
}
LIMIT 5
```

Superlative. Q: Which company has the most employees?

```
SELECT ?company ?employees
WHERE {
  ?company a dbo:Company .
  ?company dbo:numberOfEmployees ?employees .
}
ORDER BY DESC(?employees) ?company
LIMIT 1
```

Ordinal. Q: What is the founding year of (Company: Airbus)?

```
SELECT ?year
WHERE {
  VALUES ?company {
    ↪ <http://dbpedia.org/resource/Airbus> }
  ?company a dbo:Company .
```

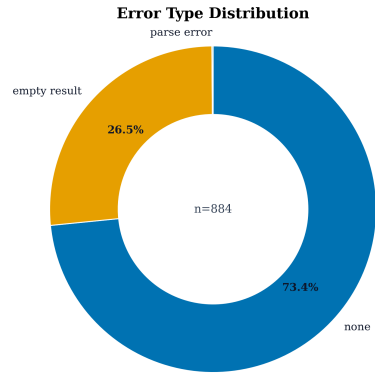


Figure 8: Error type distribution across phases.

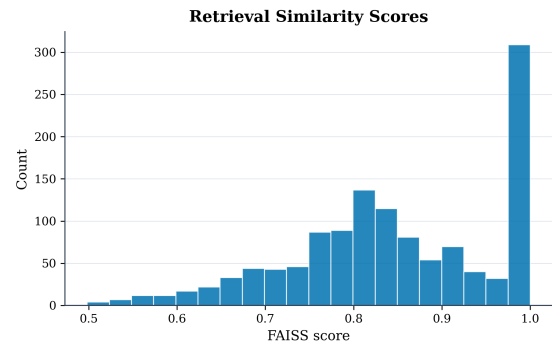


Figure 9: Distribution of retrieval similarity scores.

```
?company dbo:foundingYear ?year .
}
LIMIT 1
```

Multi-hop. Q: Which location contains companies that have a key person?

```
SELECT DISTINCT ?location
WHERE {
  ?company a dbo:Company .
  ?company dbo:keyPerson ?person .
  ?person rdf:type foaf:Person .
  ?company dbo:location ?location .
  ?location a gn:Feature .
}
LIMIT 5
```

Intersection. Q: Which companies are located in (Location: California) and are in the (Industry: Software)?

```
SELECT DISTINCT ?company
WHERE {
  ?company a dbo:Company .
  ?company dbo:location
    ↪ <http://dbpedia.org/resource/California> .
  ?company dbo:industry
    ↪ <http://dbpedia.org/resource/Software> .
}
LIMIT 5
```

Difference. Q: Which companies are located in (Location: California) but not in (Location: Texas)?

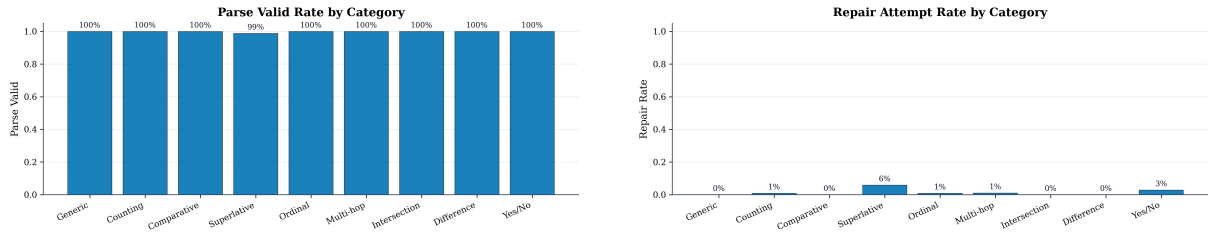


Figure 10: (Left) Parse validity by category. (Right) Repair rate by category.

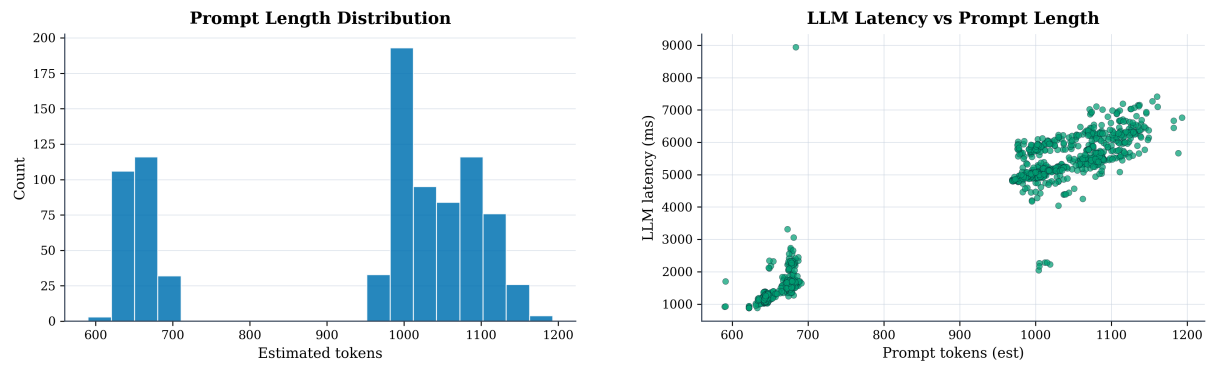


Figure 11: Prompt length distribution and latency vs. prompt length.

```
SELECT DISTINCT ?company
WHERE {
  ?company a dbo:Company .
  ?company dbo:location
    ↪ <http://dbpedia.org/resource/California> .
  FILTER NOT EXISTS {
    ?company dbo:location
      ↪ <http://dbpedia.org/resource/Texas> .
  }
}
LIMIT 5
```

Yes/No. Q: Is (Company: Facebook) located in (Location: California)?

```
ASK {
  <http://dbpedia.org/resource/Facebook> a
    ↪ dbo:Company ;
    dbo:location
      ↪ <http://dbpedia.org/resource/California>
      ↪ .
}
```